

DESAFIO PRÁTICO | FICHEIROS E EXCEÇÕES

Leitura, escrita, registos e tratamento de erros

Tempo sugerido: 40 minutos | Sem executar código | Justifica as respostas

Nome: _____ Turma: _____ Data: _____

1 MODOS DE ABERTURA

Um ficheiro dados.txt já contém informação.

a) Escolhe o modo: ler; acrescentar sem apagar; substituir o conteúdo.

b) O que acontece ao abrir um ficheiro inexistente com r? E com w?

c) Qual é a vantagem de usar with?

2 LEITURA E FIM DO FICHEIRO

nomes.txt contém duas linhas, ambas terminadas com mudança de linha: Ana e Rui.

```
with open('nomes.txt',
          encoding='utf-8') as f:
    a = f.readline()
    b = f.readline()
    c = f.readline()
```

a) Indica a, b e c, incluindo \n.

b) Qual é a diferença entre '\n' e ''?

3 ESCRITA DE NÚMEROS

```
nota = 15.5
with open('notas.txt', 'w') as f:
    f.write(nota)
```

a) Explica o erro e corrige a escrita para guardar a nota numa linha.

b) Que modo permite acrescentar outra nota sem apagar a anterior?

4 FLUXO DAS EXCEÇÕES

```
try:
    n = int(texto)
except ValueError:
    print('Inválido')
else:
    print('Aceite:', n)
finally:
    print('Fim')
```

Indica a saída para texto = '25' e para texto = 'abc'.

5 DIAGNÓSTICO E REGISTOS

a) Identifica a exceção de cada operação:

```
int('abc')
open('ausente.txt', 'r')
10 / 0
```

Assume que ausente.txt não existe.

b) Um registo de aluno ocupa três linhas: número, nome e nota. Por que deves verificar se as três foram lidas antes de o guardar?

6 DESAFIO INTEGRADOR

Lê notas.txt, com uma nota numérica válida por linha. Mostra a média ou **Sem notas** se estiver vazio. Se não existir, mostra **Ficheiro inexistente**.

a) Escreve o programa com with e try/except.

b) Testa um ficheiro com 10, 14 e 18 (uma nota por linha) e um ficheiro vazio.

DESAFIO PRÁTICO | FICHEIROS E EXCEÇÕES

Leitura, escrita, registos e tratamento de erros

SOLUÇÕES

Compara os resultados e verifica o raciocínio.

1 MODOS DE ABERTURA

- a) Ler: **r**; acrescentar: **a**; substituir: **w**.
- b) Com **r**, ocorre **FileNotFoundError**. Com **w**, o ficheiro é criado, se o caminho e as permissões o permitirem.
- Atenção: se já existir, **w** apaga o conteúdo quando abre o ficheiro.
- c) **with** fecha automaticamente o ficheiro ao sair do bloco, incluindo quando ocorre uma exceção.

2 LEITURA E FIM DO FICHEIRO

```
a = 'Ana\n'  
b = 'Rui\n'  
c = ''
```

- b) **'\n'** representa uma linha vazia que contém uma mudança de linha. **''** é a string vazia devolvida por **readline** no fim do ficheiro.
- Não confundir uma linha em branco com o fim do ficheiro.

3 ESCRITA DE NÚMEROS

- a) **write**, num ficheiro de texto, espera uma string. Um float provoca **TypeError**.

```
nota = 15.5  
with open('notas.txt', 'w',  
          encoding='utf-8') as f:  
    f.write(str(nota) + '\n')
```

- b) Usar o modo **a** para acrescentar. **\n** garante a mudança de linha entre notas.

4 FLUXO DAS EXCEÇÕES

Com texto = **'25'**:

```
Aceite: 25  
Fim
```

Com texto = **'abc'**:

```
Inválido  
Fim
```

else executa quando o **try** termina sem exceção. **finally** executa nos dois casos apresentados.

5 DIAGNÓSTICO E REGISTOS

- a) Pela ordem:
ValueError: conversão inválida.
FileNotFoundError: ficheiro inexistente.
ZeroDivisionError: divisão por zero.
- b) O ficheiro pode terminar a meio de um registo. Sem verificar as três linhas, o programa pode aceitar dados incompletos ou falhar durante a conversão.
- Depois, é necessário validar os tipos e as regras, como uma nota entre 0 e 20.

6 DESAFIO INTEGRADOR

```
try:  
    total = 0.0  
    q = 0  
    with open('notas.txt',  
              encoding='utf-8') as f:  
        for linha in f:  
            total += float(linha)  
            q += 1  
    if q:  
        print(total / q)  
    else:  
        print('Sem notas')  
except FileNotFoundError:  
    print('Ficheiro inexistente')
```

Testes: 10, 14, 18: média 14.0. Vazio: Sem notas.
Pressupõem-se linhas numéricas válidas.